

1. **Page lists:** Each partition owns its isolated slice of physical memory, so it maintains its own free, zero, standby, and modified page lists.
2. **System process:** Each partition creates its own minimal system process called “PartitionSystem.” This process provides the address space to map executables during load as well as housing per-partition system threads.
3. **System threads:** Fundamental memory management threads such as the zero page thread, the working set manager thread, the modified and mapped page writer threads are all created per-partition. In addition, other components such as the cache manager we will discuss in Sec. 11.6 also maintain per-partition threads. Finally, each partition has its dedicated system thread pool such that kernel components can queue work to it without worrying about contention from other workloads.
4. **Pagefiles:** Each partition has its own set of pagefiles and associated modified page writer thread. This is critical for maintaining its own commit.
5. **Resource tracking:** Each partition holds its own memory management resources such as commit and available memory to independently drive policies such as working set trimming and pagefile writing.

Notably, a memory partition does not include its own PFN database. Instead, it maintains a data structure describing the memory ranges it is responsible for and uses the system global PFN database entries. Also, most threads and data structures are initialized on demand. For example, the modified page writer thread is not necessary until a pagefile is created in the partition.

All in all, memory management is a highly complex executive component with many data structures, algorithms, and heuristics. It attempts to be largely self-tuning, but there are also many knobs that administrators can tweak to affect system performance. A number of these knobs and the associated counters can be viewed using tools in the various tool kits mentioned earlier. Probably the most important thing to remember here is that memory management in real systems is a lot more than just one simple page replacement algorithm like clock or aging.

11.6 CACHING IN WINDOWS

The Windows cache improves the performance of file systems by keeping recently and frequently used regions of files in memory. Rather than cache physical addressed blocks from the disk, the cache manager manages virtually addressed

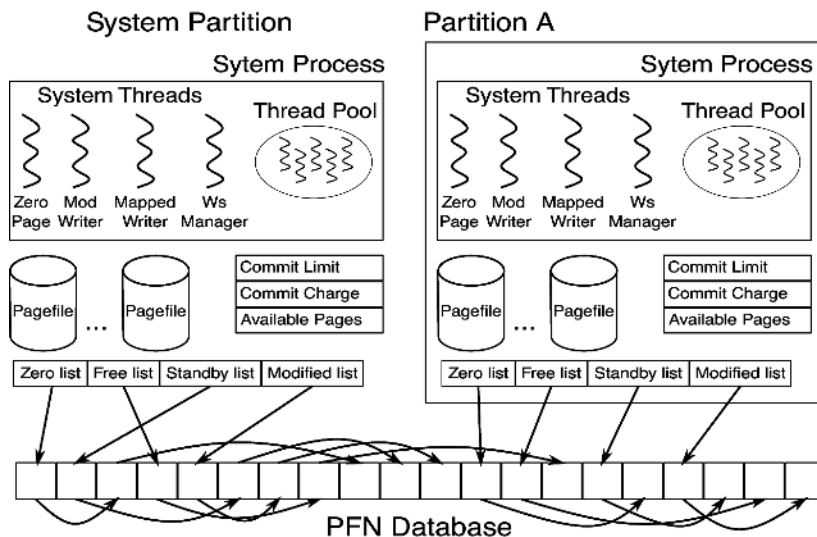


Figure 11-39. Memory partition data structures.

blocks, that is, regions of files. This approach fits well with the structure of the native NT File System (NTFS), as we will see in Sec. 11.8. NTFS stores all of its data as files, including the file-system metadata.

The cached regions of files are called *views* because they represent regions of kernel virtual addresses that are mapped onto file-system files. Thus, the actual management of the physical memory in the cache is provided by the memory manager. The role of the cache manager is to manage the use of kernel virtual addresses for views, arrange with the memory manager to pin pages in physical memory, and provide interfaces for the file systems.

The Windows cache-manager facilities are shared among all the file systems. Because the cache is virtually addressed according to individual files, the cache manager is easily able to perform read-ahead on a per-file basis. Requests to access cached data come from each file system. Virtual caching is convenient because the file systems do not have to first translate file offsets into physical block numbers before requesting a cached file page. Instead, the translation happens later when the memory manager calls the file system to access the page on disk.

Besides management of the kernel virtual address and physical memory resources used for caching, the cache manager also has to coordinate with file systems regarding issues like coherency of views, flushing to disk, and correct maintenance of the end-of-file marks—particularly as files expand. One of the most difficult aspects of a file to manage between the file system, the cache manager, and the memory manager is the offset of the last byte in the file, called the **Valid-DataLength**. If a program writes past the end of the file, the blocks that were

skipped have to be filled with zeros, and for security reasons it is critical that the `ValidDataLength` recorded in the file metadata not allow access to uninitialized blocks, so the zero blocks have to be written to disk before the metadata is updated with the new length. While it is expected that if the system crashes, some of the blocks in the file might not have been updated from memory, it is not acceptable that some of the blocks might contain data previously belonging to other files.

Let us now examine how the cache manager works. When a file is referenced, the cache manager maps a 256-KB chunk of kernel virtual address space onto the file. If the file is larger than 256 KB, only a portion of the file is mapped at a time. If the cache manager runs out of 256-KB chunks of virtual address space, it must unmap an old file before mapping in a new one. Once a file is mapped, the cache manager can satisfy requests for its blocks by just copying from kernel virtual address space to the user buffer. If the block to be copied is not in physical memory, a page fault will occur and the memory manager will satisfy the fault in the usual way. The cache manager is not even aware of whether the block was in memory or not. The copy always succeeds.

The cache manager has various heuristics for detecting file access patterns. For example, when it detects a sequential access pattern, it starts performing **read-ahead** on behalf of the application such that data is ready by the time the application issues its I/O. This is very similar to the prefetching performed by the memory manager and uses the same underlying memory manager APIs.

Another important background operation the cache manager performs is **write-behind**. When dirty data accumulates in the cache manager's virtual address space, it starts proactively writing out the dirty data to disk to minimize the amount of lost data if, for example, the power goes out. Applications can always use the `FlushFileBuffers` Win32 API to flush all dirty data to disk; write-behind is a secondary measure. Another important benefit of write-behind is that the underlying pages can be more quickly reclaimed by the memory manager if available memory starts running low.

The cache manager also works for pages that are mapped into virtual memory and accessed with pointers rather than being copied between kernel and user-mode buffers. When a thread accesses a virtual address mapped to a file and a page fault occurs, the memory manager may in many cases be able to satisfy the access as a soft fault. It does not need to access the disk, since it finds that the page is already in physical memory because it is mapped by the cache manager.

11.7 INPUT/OUTPUT IN WINDOWS

The goals of the Windows I/O manager are to provide a fundamentally extensive and flexible framework for efficiently handling a very wide variety of I/O devices and services, support automatic device discovery and driver installation (plug and play) and efficient power management for devices and the CPU—all using a